refait le prompt pour que Claude code puisse realisé l'application avec API Deepseek

Tu peux donner ce prompt à Claude Code pour qu'il te génère l'architecture + code, en le laissant ensuite itérer module par module.

Prompt Claude Code – T-Cardio Pro + DeepSeek

Tu es **Claude Code**, un ingénieur logiciel senior full-stack chargé de concevoir et développer la plateforme T-Cardio Pro, en utilisant **DeepSeek** comme moteur IA d'assistance médicale (NON diagnostique).

1. Contexte fonctionnel & médical

T-Cardio Pro est une plateforme intelligente de suivi cardiovasculaire et d'hypertension avec :

- Espace patient (mobile + web)
- Espace médecin/cardiologue (web + mobile simplifié)
- Espace admin (web)
- IA d'analyse et de triage NON prescriptive (DeepSeek API)

Contraintes médicales et légales :

- L'IA n'est PAS un dispositif de diagnostic autonome.
- L'IA ne prescrit JAMAIS de traitement, ne donne pas de dosages, ne modifie pas de traitement.
- L'IA fournit : analyses, tendances, probabilités, niveaux de risque, alertes argumentées, résumés patient & médecin.
- Toute décision clinique appartient au médecin humain.
- La plateforme doit être juridiquement défendable (audit, traçabilité, logs IA).

2. Stack technique OBLIGATOIRE

- **Frontend**
  - Mobile : Flutter ou React Native (au choix, mais une seule stack mobile).
  - Web : React + TypeScript.
- **Backend Core**

    - Node.js + NestJS (architecture modulaire).
    - API REST sécurisée (JSON).
- **Base de données**
    - PostgreSQL (ou Supabase) avec migrations (Prisma ou TypeORM).
- **IA**
    - Microservice “ai-engine” (NestJS ou FastAPI) qui appelle **DeepSeek API**.
    - Le frontend n’appelle jamais directement DeepSeek.
    - Prompt système verrouillé + versionnement des prompts et des réponses.
- **Sécurité**
    - Authentification JWT + refresh tokens.
    - RBAC (Patient, Médecin, Cardiologue, Admin).
    - Chiffrement des données sensibles.
    - Audit logs (accès aux dossiers, appels IA, téléconsultations, rapports).

3. Modules fonctionnels à implémenter

MODULE A — Authentification & rôles

- Inscription patient.
- Inscription médecin (validation par admin avant activation).
- Connexion, refresh token, logout.
- Rôles : PATIENT, MEDECIN, CARDIOLOGUE, ADMIN.
- Guards NestJS pour restreindre l’accès par rôle.

MODULE B — Profil patient

- Informations de base : âge, sexe, taille, poids, IMC (calculable).
- Statut médical : STANDARD, HYPERTENDU, POST_AVC, DIABETIQUE, AUTRE.
- Liaison éventuelle à un médecin référent.

MODULE C — Mesures tension

- Création mesure : systolique, diastolique, pouls, date/heure, commentaire.
- Historique complet paginé, filtrable par période (7/30/90 jours).
- Préparation future pour intégration Bluetooth (champ `source` MANUEL/BLUETOOTH).

MODULE D — Visualisation & analytique

- Endpoints de synthèse :
  - Moyennes mobiles (7/30/90 jours).
  - Variabilité tensionnelle.
  - Comparaison matin/soir.
- API pour alimenter des graphes (courbes SYS/DIA, codes couleurs risque).

MODULE E — IA T-Cardio Pro (DeepSeek)

Microservice IA avec :

- Endpoint interne (appelé par backend Core) :
  - Input : profil patient + dernier historique de mesures (par ex. 30/60/90 jours) + contexte (observance, commentaires).
  - Output structuré JSON :
    - `risk_level`: FAIBLE / MODERE / ELEVE / CRITIQUE
    - `projections`: { `30d`, `60d`, `90d` }
    - `alerts`: liste d'alertes (type, sévérité, justification)
    - `patient_summary`: texte simple pour patient
    - `doctor_summary`: texte clinique pour médecin
- Le microservice construit un prompt pour DeepSeek :
  - **system_prompt** : règles médicales strictes (pas de diagnostic, pas de prescription, style, structure).
  - **user_prompt** : JSON des données patient + mesures.
- Il appelle l'API DeepSeek, parse la réponse, la transforme en JSON strict.
- Il logge : input, output, modèle, version de prompt, timestamps, dans la table `ai_analyses`.

IMPORTANT :

- Le backend Core doit filtrer toute sortie IA qui contiendrait des prescriptions explicites (regex simple) et les bloquer / nettoyer.

MODULE F — Mode urgence

- Règle dans backend Core : si `systolic >= 180` ou `diastolic >= 120`, créer automatiquement une alerte critique.
- Génération d'un événement URGENCE :
  - Notification au médecin référent (endpoint + hook pour futur SMS/email).
  - Recommandation de consultation urgente (dans les messages patient).
  - Journalisation dans `alerts` + `audit_logs`.

MODULE G — Tableau de bord médecin

- Liste des patients suivis avec : niveau de risque, dernière mesure, dernières alertes.
- Accès dossier patient :
  - Courbes tensionnelles.
  - Dernières analyses IA.
  - Historique d'alertes.

MODULE H — Téléconsultation

- Modèle simple de téléconsultation (MVP) :
  - Planifier une téléconsultation (date/heure, patient, médecin).
  - Statuts : PLANNED, ACTIVE, ENDED, CANCELLED.
  - Chat texto sécurisé (API + WebSocket).
  - Intégration WebRTC ou placeholder avec un champ `rtc_room_id` à connecter à un service tiers plus tard.

MODULE I — Rapports PDF

- Génération de PDF (backend) contenant :
  - Identité patient.
  - Graphiques (via images ou lien).
  - Synthèse IA (patient + médecin).
  - Notes médecin.
- Stockage dans S3 ou équivalent.
- Endpoints de téléchargement sécurisés (autorisations par rôle).

MODULE J — Administration

- Validation manuelle des comptes médecins.
- Gestion des utilisateurs (activation/désactivation).
- Statistiques globales anonymisées (nombre mesures, analyses IA, alertes).
- Paramétrage des seuils IA (par ex. tension critique, niveaux de risque).

4. Modèle de données (PostgreSQL)

Demande :

- Proposer un schéma complet (Prisma ou TypeORM) comprenant au minimum :
  - `users`, `doctors`, `patients`, `patient_doctor_links`
  - `bp_measurements`

- `ai_analyses`
- `alerts`
- `teleconsultations`, `teleconsultation_messages`
- `medical_notes`
- `reports`
- `audit_logs`

5. API REST à générer

Demande à Claude :

1. Lister toutes les routes REST du backend Core (NestJS) par module, avec :
   - Méthode, URL, rôle(s) autorisés, payload (DTO), structure de réponse.
2. Proposer la structure des modules NestJS (dossiers, services, controllers, guards, interceptors).
3. Proposer le contrat d'API entre backend Core et microservice IA DeepSeek (request/response JSON).

6. Intégration DeepSeek

Demande :

- Définir un **client HTTP** pour DeepSeek avec :
  - URL, headers, gestion de la clé API, timeouts, retries basiques.
- Proposer :
  - `system_prompt` complet pour T-Cardio Pro IA (respect des règles médicales).
  - Format de réponse imposé à DeepSeek (JSON strict).
- Générer le code du service IA :
  - Fonction `analyzePatientData(input: PatientAiInput): Promise<AiAnalysisResult>`
  - Gestion d'erreur si DeepSeek répond mal formé / timeout.

7. UX / UI

Demande :

- Proposer la structure d'écrans pour :
  - App patient (mobile + web) : onboarding, dashboard, saisie mesure, historique, vue IA, urgences, téléconsultation.
  - App médecin (web) : dashboard, dossier patient, vue IA, téléconsultation, rapports.
  - App admin (web) : gestion médecins, stats, paramètres IA.

- Générer des composants React/React Native de base (squelettes) connectés aux endpoints.

8. Ce que tu dois produire en priorité

1. Architecture NestJS (modules, dossier, configuration).
2. Schéma DB (Prisma ou TypeORM).
3. Liste détaillée des endpoints REST.
4. Contrat du microservice IA DeepSeek (code + types).
5. Prompt système DeepSeek pour T-Cardio Pro.
6. Squelettes de composants frontend (React + React Native ou Flutter) appelant les endpoints.

Commence par :

1. proposer le schéma DB,
2. puis l'architecture NestJS (modules + fichiers),
3. puis les endpoints,
4. puis le microservice IA DeepSeek (contrat + prompt),
5. puis les composants frontend.